

CR-005: Global Session Expiry and Friendly Re-Login

1. Change Request Details

Change Request ID: CR-005

Title: Global Session Expiry and Friendly Re-Login

Priority: High

Difficulty: Easy

Change Type: Frontend Authentication Change

2. Business Requirement

Users must receive a clear and user-friendly message when their authentication session expires or becomes invalid instead of seeing confusing API errors.

The frontend must provide global handling for HTTP 401 Unauthorized responses using an Axios response interceptor. When an authenticated user's session expires, the application must safely clear the stored authentication information, redirect the user to the Login page, and inform them that their session has expired.

The page the user was attempting to access must be preserved so that, after successful re-authentication, the user can be returned to the original page.

The implementation must distinguish between expired sessions, invalid login credentials, network failures, and access-denied responses.

3. Current Behaviour

- JWT authentication tokens expire after a fixed period.
- The frontend does not currently have a global Axios response interceptor for authentication failures.
- Expired or invalid tokens may remain in local storage.
- HTTP 401 responses may result in confusing API errors being shown to the user.
- There is no consistent global redirect to the Login page when a session expires.
- The Login page does not consistently display a dedicated session-expired message.
- The login request itself may potentially be affected if global 401 handling is applied without excluding the login endpoint.
- The originally requested page may not be preserved when the user is redirected to Login.
- Multiple simultaneous API requests may produce multiple 401 responses and potentially trigger multiple redirects.
- Network errors must be distinguished from authentication failures.

- HTTP 403 responses must not automatically clear authentication data or log the user out.
- Users accessing a page without the required role must receive an access-denied message rather than being logged out.

4. Requested Change

Improve frontend authentication handling by implementing global session-expiry detection and a friendly re-login flow.

The application must:

- Add a global Axios response interceptor.
- Detect HTTP 401 Unauthorized responses.
- Exclude the Login request from session-expiry handling.
- Clear the stored authentication token when an authenticated session expires or becomes invalid.
- Clear the stored user information when an authenticated session expires or becomes invalid.
- Redirect the user to `/login`.
- Preserve the page the user was attempting to access.
- Display the following message on the Login page when the session has expired:

Your session has expired. Please log in again.

- Continue displaying the following message for incorrect login credentials:

Invalid username or password

- Prevent the Login request itself from triggering the session-expired redirect.
- Ensure multiple simultaneous 401 responses result in only one redirect.
- Ensure network errors do not clear authentication information.
- Ensure HTTP 403 responses do not automatically log the user out.
- Display an access-denied message when a user is authenticated but does not have the required role.
- Return the user to the originally requested page after successful login.

5. Scope

This change applies to frontend authentication, session handling, protected routes, login behaviour, and related navigation feedback.

The change includes:

- Global Axios 401 response handling.
- Detection of expired or invalid authentication sessions.
- Clearing the stored authentication token.
- Clearing the stored user information.
- Redirecting expired sessions to `/login`.
- Preserving the originally requested page.
- Displaying a session-expired message.
- Preserving the existing invalid-login-credentials message.
- Excluding the login request from session-expiry handling.
- Preventing duplicate redirects caused by simultaneous 401 responses.

- Preserving authentication data during network failures.
- Preventing automatic logout for HTTP 403 responses.
- Displaying an access-denied message for authenticated users with insufficient roles.
- Returning users to their original page after successful login.
- Maintaining existing authentication and authorization behaviour.

The change does not include:

- Changes to JWT generation or expiration configuration on the backend.
- Changes to user authentication database records.
- Changes to database schema.
- Changes to password validation logic.
- Changes to user registration functionality.
- Changes to role definitions.
- Changes to backend authorization rules.
- Introducing a new authentication API.
- Introducing a new backend session-management system.

6. Functional Requirements

6.1 Global Authentication Error Handling

FR-001: Axios Response Interceptor

The frontend must implement a global Axios response interceptor to handle authentication-related HTTP responses consistently.

The interceptor must be registered through the application's existing Axios/API service implementation.

FR-002: HTTP 401 Detection

The Axios response interceptor must detect HTTP 401 Unauthorized responses.

A 401 response from an authenticated API request must be treated as an indication that the current authentication session is no longer valid.

FR-003: Authentication Data Clearing

When an authenticated API request receives a 401 response, the frontend must clear:

- The stored authentication token.
- The stored authenticated user information.

The implementation must use the application's existing local storage keys or authentication storage conventions.

FR-004: Login Redirect

After handling an authenticated 401 response, the user must be redirected to:

`/login`

The redirect must occur without requiring the user to manually navigate to the Login page.

FR-005: Session Expired Message

When the user is redirected to Login because of an expired or invalid session, the Login page must display:

Your session has expired. Please log in again.

The message must only be displayed when the Login page was reached because of session-expiry handling.

6.2 Login Request Handling

FR-006: Login Request Exclusion

The login request itself must not trigger the session-expired redirect when it returns HTTP 401.

A 401 response from the login endpoint must continue to be treated as an invalid-login-credentials failure.

FR-007: Invalid Login Credentials

When the user enters incorrect login credentials, the Login page must continue to display:

Invalid username or password

The session-expired message must not be displayed for a normal failed login attempt.

FR-008: Login Error Separation

The implementation must distinguish between:

- A 401 received while attempting to log in.
- A 401 received while using an authenticated session.

Only the second case must trigger global session-expiry handling.

6.3 Original Page Preservation

FR-009: Preserve Requested Page

When an authenticated user receives a 401 response and is redirected to Login, the frontend must preserve the page or route that the user was attempting to access.

The preserved destination may include the relevant route and required navigation information according to the existing application routing structure.

FR-010: Return After Successful Login

After the user successfully logs in following a session-expiry redirect, the application must return the user to the originally requested page.

If no original page is available, the application may use the application's existing default post-login destination.

FR-011: Authentication State Restoration

After successful re-login, the newly issued authentication token and user information must be stored using the existing authentication mechanism before navigating to the preserved destination.

6.4 Multiple Simultaneous 401 Responses

FR-012: Single Redirect

Multiple simultaneous API requests returning HTTP 401 must result in only one session-expired redirect.

The application must prevent repeated navigation to `/login` caused by multiple concurrent 401 responses.

FR-013: Single Session Cleanup

Multiple simultaneous 401 responses must not cause inconsistent authentication state.

Clearing the stored token and user information must remain safe when multiple requests fail at approximately the same time.

FR-014: Redirect Guard

The implementation must include appropriate state or logic to prevent duplicate session-expiry redirects while the user is already being redirected to Login.

6.5 Network Error Handling

FR-015: Network Error Preservation

Network errors must not be treated as authentication failures.

A network failure must not automatically:

- Clear the authentication token.
- Clear the stored user.
- Redirect the user to Login.

FR-016: 401 Versus Network Failure

The implementation must distinguish an actual HTTP 401 response from an error where no HTTP response is available.

Only an actual HTTP 401 response must trigger session-expiry handling.

6.6 HTTP 403 Handling

FR-017: Preserve Authentication on 403

An HTTP 403 Forbidden response must not automatically log the user out.

The authentication token and stored user information must remain available unless another existing application rule explicitly requires different handling.

FR-018: Access Denied Message

When an authenticated user attempts to access a resource for which they do not have the required role or permission, the application must display an appropriate access-denied message.

For example:

Access denied. You do not have permission to access this page.

The exact wording may follow the application's existing UI conventions.

FR-019: Wrong Role Remains Logged In

A user with a valid authentication session but an incorrect role must remain logged in.

A role or authorization failure must not be treated as an expired session.

6.7 Protected Route Behaviour

FR-020: Protected Route Compatibility

Existing ProtectedRoute behaviour must continue to work correctly after the global Axios interceptor is introduced.

FR-021: Unauthenticated Access

Users who are not authenticated and attempt to access protected pages must continue to follow the application's existing authentication flow.

FR-022: Authorized Access

Users with valid authentication and the required role must continue to access permitted pages normally.

FR-023: Unauthorized Role Access

Users with valid authentication but an incorrect role must receive the appropriate access-denied behaviour without being logged out.

6.8 Authentication Data Handling

FR-024: Token Removal

When a valid authenticated request receives a 401, the existing stored authentication token must be removed.

FR-025: User Removal

When a valid authenticated request receives a 401, the existing stored user information must be removed.

FR-026: No Unnecessary Data Removal

Authentication data must not be cleared for:

- Network failures.
- HTTP 403 responses.
- Normal login failures.
- Other unrelated HTTP errors.

6.9 Existing Functionality Requirements

FR-027: Existing Login Functionality

Existing successful login functionality must continue to work correctly.

FR-028: Existing Logout Functionality

Existing logout functionality must continue to work correctly.

FR-029: Existing Authorization

Existing role-based authorization behaviour must remain unchanged unless required to display the appropriate access-denied message.

FR-030: Existing Navigation

Existing application navigation must continue to work after introducing the global interceptor.

7. Security Requirements

SEC-001: Existing Authentication

Existing authentication requirements and mechanisms must remain unchanged unless required to implement session-expiry handling.

SEC-002: Authentication Data Clearing

When an authenticated session is determined to be invalid through a 401 response, the frontend must remove the stored authentication token and user information.

SEC-003: Student/User Data Isolation

The session-expiry implementation must not expose or retrieve data belonging to another authenticated user.

SEC-004: Existing Authorization

Existing role-based authorization and access-control behaviour must be preserved.

SEC-005: No Authentication Bypass

The session-expiry handling must not provide a mechanism for an unauthenticated user to access protected pages.

SEC-006: Protected Destination Handling

The preserved destination must only be used for navigation after successful authentication. It must not bypass existing route protection or authorization checks.

SEC-007: Network Failure Safety

Network failures must not cause authentication information to be deleted because of the absence of an HTTP response.

8. Configuration / Potentially Affected Components

Frontend

Potentially affected areas include:

- `frontend/src/services/api.js`
- `frontend/src/services/auth.js`
- `frontend/src/pages/Login.jsx`
- `frontend/src/components/ProtectedRoute.jsx`
- `frontend/src/components/Navbar.jsx`
- `frontend/src/App.css`

The exact files may vary depending on the existing implementation.

Backend

No compulsory backend changes are expected.

Existing authentication and authorization APIs should continue to operate as currently implemented.

API

No new API endpoint is required.

The existing API responses, particularly HTTP 401 and HTTP 403 responses, must be handled by the frontend according to this change request.

Database

No database changes are required.

Existing user, role, and authentication-related data must continue to be used.

Expected / Potentially Affected Files

The following files are examples of expected or potentially affected files, not a mandatory list of files that must all be modified:

- `frontend/src/services/api.js`
- `frontend/src/services/auth.js`
- `frontend/src/pages/Login.jsx`
- `frontend/src/components/ProtectedRoute.jsx`
- `frontend/src/components/Navbar.jsx`
- `frontend/src/App.css`

Students must perform an impact analysis before implementation.

If additional files are modified, they must be identified and justified in the Implementation Report.

9. Constraints / Assumptions

- Existing authentication functionality should continue to work.
- Existing JWT authentication should continue to be used.
- No compulsory backend changes are required.
- No database changes are required.
- The existing Axios/API service should be reused where possible.
- HTTP 401 responses must be handled globally for authenticated API requests.
- The Login request must be excluded from global session-expiry handling.
- Incorrect login credentials must continue to display `Invalid username or password`.
- Session expiry must display `Your session has expired. Please log in again`.
- Authentication data must be cleared when an authenticated session receives a 401 response.
- Network errors must not clear authentication data.
- HTTP 403 responses must not automatically log the user out.
- Users with incorrect roles must remain authenticated.
- The originally requested page must be preserved during session-expiry redirection.
- Successful re-login must return the user to the preserved page.
- Multiple simultaneous 401 responses must result in only one redirect.
- Existing protected-route and authorization behaviour must continue to work.
- The implementation must not introduce an authentication bypass.
- Existing local storage conventions should be preserved.
- No additional authentication API is required.

10. Acceptance Criteria

AC-001: Expired Session Detection

When an authenticated API request receives an HTTP 401 response because the session is expired or invalid, the frontend handles the response through the global Axios interceptor.

AC-002: Authentication Data Removal

After an authenticated 401 response, the stored authentication token is removed from local storage.

AC-003: Stored User Removal

After an authenticated 401 response, the stored user information is removed from local storage.

AC-004: Login Redirect

An authenticated 401 response redirects the user to `/login`.

AC-005: Session Expired Message

After an expired session causes a Login redirect, the Login page displays:

Your session has expired. Please log in again.

AC-006: Correct Login Failure Message

When the user enters incorrect login credentials, the Login page displays:

Invalid username or password

The session-expired message is not displayed.

AC-007: Login Request Exclusion

A 401 response from the Login request does not trigger the global session-expired redirect.

AC-008: Requested Page Preservation

When a user's session expires while accessing a protected page, the application preserves the originally requested page.

AC-009: Return After Login

After successfully logging in following session expiry, the user is returned to the originally requested page.

AC-010: Single Redirect

Multiple simultaneous HTTP 401 responses cause only one redirect to `/login`.

AC-011: No Duplicate Navigation

The application does not repeatedly navigate to `/login` when multiple API requests fail with 401 during the same session-expiry event.

AC-012: Network Error Preservation

A network error without an HTTP response does not clear the authentication token.

AC-013: Network Error User Preservation

A network error without an HTTP response does not clear the stored user information.

AC-014: Network Error No Logout

A network failure does not automatically redirect the user to Login.

AC-015: HTTP 403 Preservation

An HTTP 403 response does not automatically clear authentication data or log the user out.

AC-016: Wrong Role Access

An authenticated user attempting to access a page requiring a different role receives an appropriate access-denied message.

AC-017: Wrong Role Remains Logged In

A user with a valid session but an incorrect role remains authenticated after receiving an access-denied response.

AC-018: Existing Login Functionality

Successful login continues to work correctly.

AC-019: Existing Protected Routes

Existing protected routes continue to enforce authentication correctly.

AC-020: Existing Authorization

Existing role-based access control continues to function correctly.

AC-021: Authentication Data Safety

Authentication information is not removed for normal login failures, network failures, HTTP 403 responses, or unrelated errors.

AC-022: No Authentication Bypass

The session-expiry implementation does not allow unauthenticated users to access protected pages.

AC-023: Existing Navigation

Existing application navigation continues to function correctly after the Axios interceptor is introduced.

AC-024: Existing Logout

Existing logout functionality continues to work correctly.

AC-025: Session-Expired Message Only When Appropriate

The session-expired message is displayed only when the user was redirected to Login because of an expired or invalid authenticated session.

AC-026: No Backend Changes Required

The functionality can be implemented without requiring changes to the backend authentication API.

AC-027: No Database Changes

The functionality can be implemented without requiring database schema or data changes.

AC-028: Global Interceptor

HTTP 401 handling is implemented through a global Axios response interceptor rather than duplicating session-expiry logic across individual API calls.

AC-029: Multiple 401 Safety

Multiple simultaneous 401 responses do not produce inconsistent authentication state or multiple session-expiry redirects.

AC-030: CRUD / Existing Functionality

Existing application authentication, authorization, navigation, and protected-page functionality continues to work correctly after the change.

11. Expected Outcome

After implementation:

- Users receive a clear message when their authentication session expires.
- Expired or invalid authenticated sessions are detected globally through the Axios response interceptor.
- Authentication tokens are removed when an authenticated 401 response is received.
- Stored user information is removed when an authenticated 401 response is received.
- Users are redirected to `/login` after session expiry.
- The message "**Your session has expired. Please log in again.**" is displayed after an appropriate session-expiry redirect.
- Incorrect login credentials continue to display "**Invalid username or password**".
- The Login request itself does not trigger the session-expired redirect.
- The page the user was attempting to access is preserved.
- Successful re-login returns the user to the originally requested page.
- Multiple simultaneous 401 responses result in only one redirect.
- Network errors do not clear authentication information.
- HTTP 403 responses do not automatically log users out.
- Users with valid authentication but incorrect roles receive an access-denied message.
- Wrong-role users remain logged in.
- Existing authentication and authorization behaviour continues to work.
- Existing protected routes continue to function correctly.
- No authentication bypass is introduced.
- No compulsory backend changes are required.
- No database changes are required.
- Existing application functionality continues to work correctly after the authentication changes.